HPC, Nextflow, and GenPipes

Instructors: Benjamin Kaufman & Gerardo Zapata

D2R DNA TO RNA | HeDS

C3G Canadian Centre for Computational Genomics

McGill | CDSI Computational and Data Systems Institute | ISCD Institut en systèmes computationnels et de données

McGill initiative in Computational Medicine

Centre universitaire de santé McGill | McGill University Health Centre

McGill Quantitative Life Sciences | Sciences quantitatives du vivant

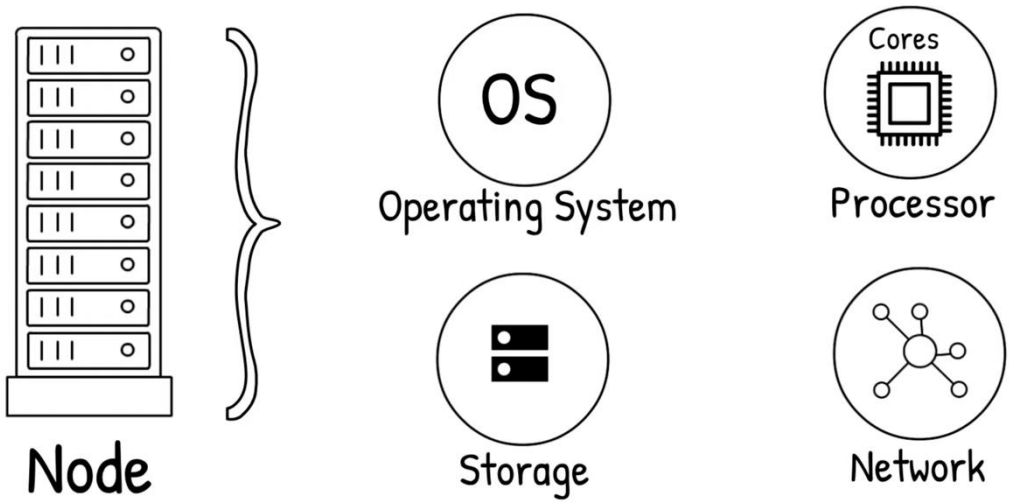
Learning objectives

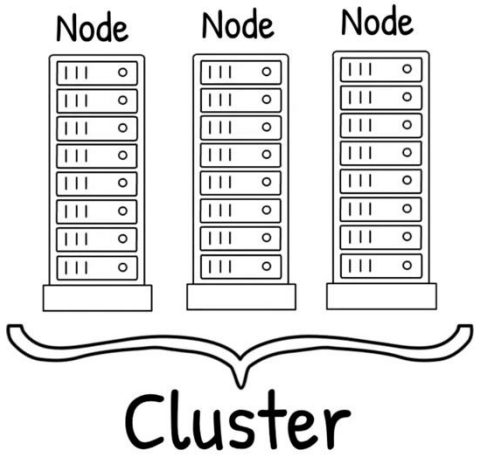
1. Learn basic high performance computer (HPC) infrastructure.

2. Get familiar to how pipelines work at the HPC of the Digital Research Alliance of Canada (DRAC).

3.Understand the basics of pre-built GenPipes/Nextflow RNA-seq pipelines using DRAC resources.

High performance computing





Adapted from Georgi Merhi

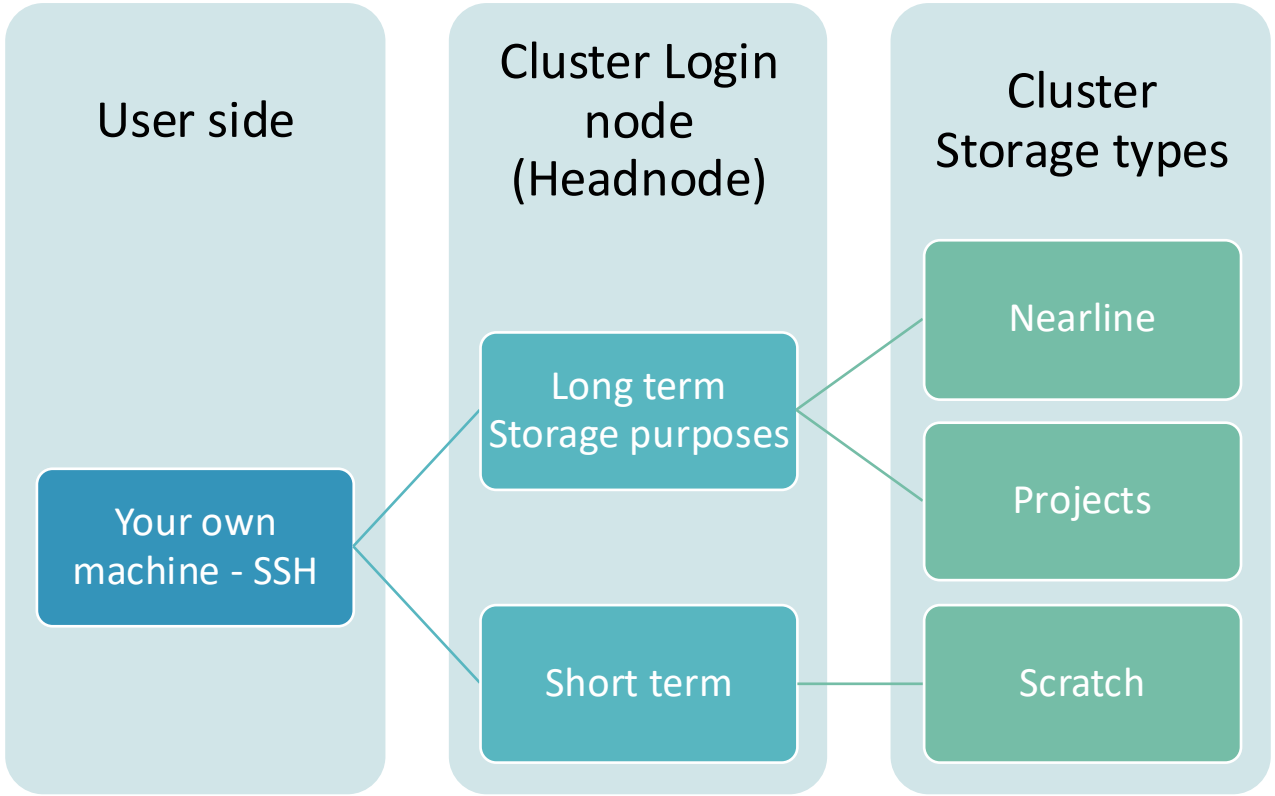
https://www.youtube.com/watch?v=nIBu1EFYmBU

Digital Research Alliance of Canada (Compute Canada)

Digital Research Alliance of Canada | Alliance de recherche numérique du Canada

Adapted from Georgi Merhi

Digital Research Alliance of Canada (Compute Canada)



Adapted from Georgi Merhi

Digital Research Alliance of Canada (Compute Canada)

Feature	Scratch Storage	Nearline Storage	Projects Storage
Purpose	Temporary, high-performance storage	Long-term, infrequently accessed data	Persistent, project-related storage
Performance	High	Moderate to Low	Moderate to High
Capacity	Large	Large	Moderate to Large
Backup	No	Yes	Yes
Purge Policy	Regular purges	Data retained long-term	Retained throughout project duration
Accessibility	**Individual user**	**Individual user**	**Shared among project members**

Adapted from Georgi Merhi

MacBook Pro 2026
RAM: 16-32GB
Storage Memory: 1TB-8TB
Terminal is locked while job runs

Digital Research Alliance of Canada

Alliance de recherche numérique du Canada

Compute Canda
RAM: 250GB-4,000GB*
Storage Memory: 50GB-40TB
Built in job scheduling system

- Overview of some important terms related to the cluster:
  - Compute Nodes → **one node = one computer**
  - Job Scheduling System → **SLURM**
  - SLURM job priority → **Fairshare**

What is the benefit of running a job through a scheduler (SLURM)?

```
[georgi26@narval1 scratch]$ cat tarring.sh
#!/bin/bash
#SBATCH --job-name=tarball_creation_GM
#SBATCH --account=ctb-shapiro
#SBATCH --nodes=1
#SBATCH --cpus-per-task=16
#SBATCH --mail-type=ALL
#SBATCH --mail-user=georgi.merhi@mail.mcgill.ca
#SBATCH --output=tarball_creation_%j.out
#SBATCH --error=tarball_creation_%j.err
#SBATCH --time=23:00:00
#SBATCH --mem=8G

# Navigate to the directory you want to archive
cd /home/georgi26/scratch/

# Create a tar.gz archive of everything in the directory
tar -czvf Georgi_all_13MAI2024_v2.tar.gz *
```


Adapted from Georgi Merhi

You can also split your job to run parallel – faster processes!

An array job internally behaves like a for loop and submits individual jobs

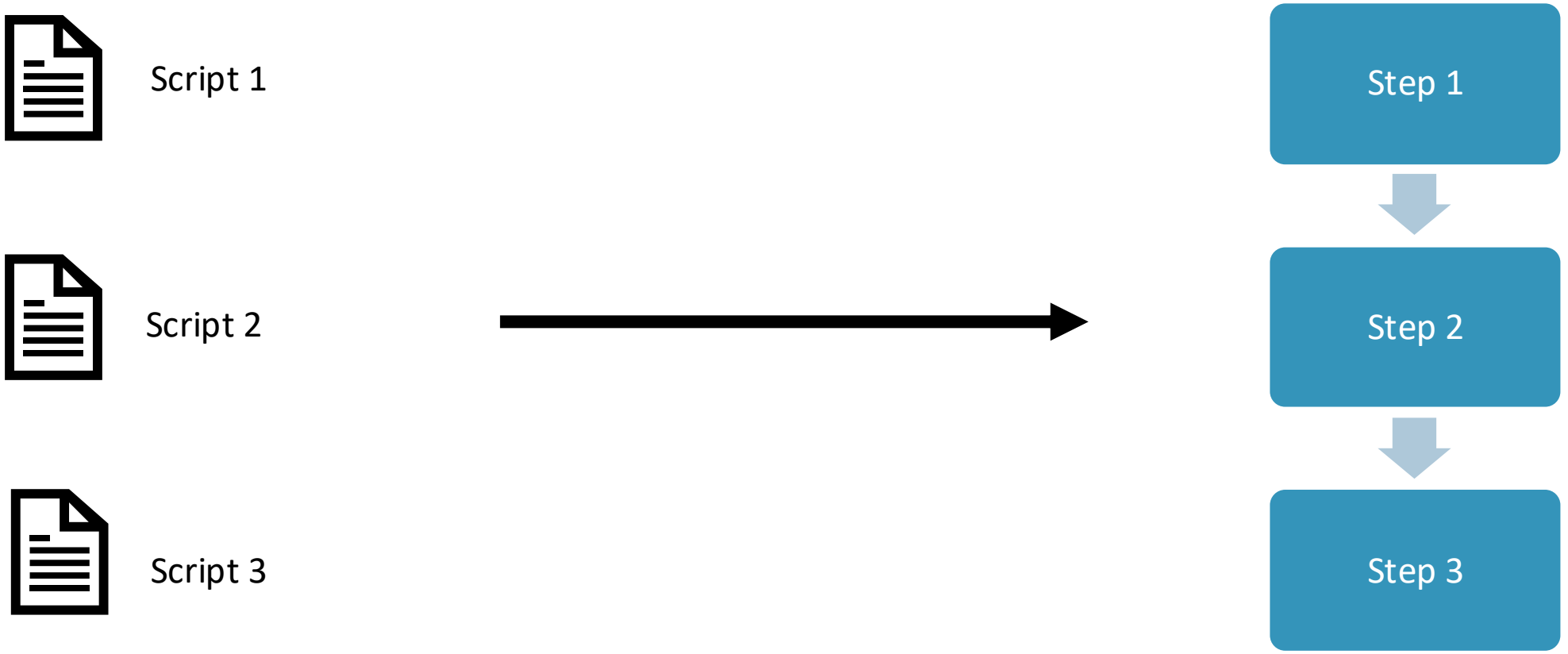
```
[georgi26@narval1 phylo_RAW]$ cat phylo_snp_2.sh
#!/bin/sh

#SBATCH --account=ctb-shapiro
#SBATCH -o /home/georgi26/scratch/phylo_RAW/output_e_o/gatk_call_variants_%j.o
#SBATCH -e /home/georgi26/scratch/phylo_RAW/output_e_o/gatk_call_variants_%j.e
#SBATCH -t 08:00:00
#SBATCH --mem=64G
#SBATCH -c 8
#SBATCH --job-name=GM_gatk_variant_call
#SBATCH --array=1000-1273 #Indicate number of isolates to call variants from.

## Calling SNPs using GATK HaplotypeCaller for phylogeny construction
# Load packages
module load StdEnv/2020 gcc/9.3.0
module load bwa
module load samtools
module load java picard
module load gatk
```


Adapted from Georgi Merhi

- A scientific workflow system predominantly used for bioinformatic data analysis
  - Easy construction of a complete pipeline and regular checkpoints
  - Highly Portable
  - Parallelizable
  - Prioritize desired outputs while minimizing intermediate file management

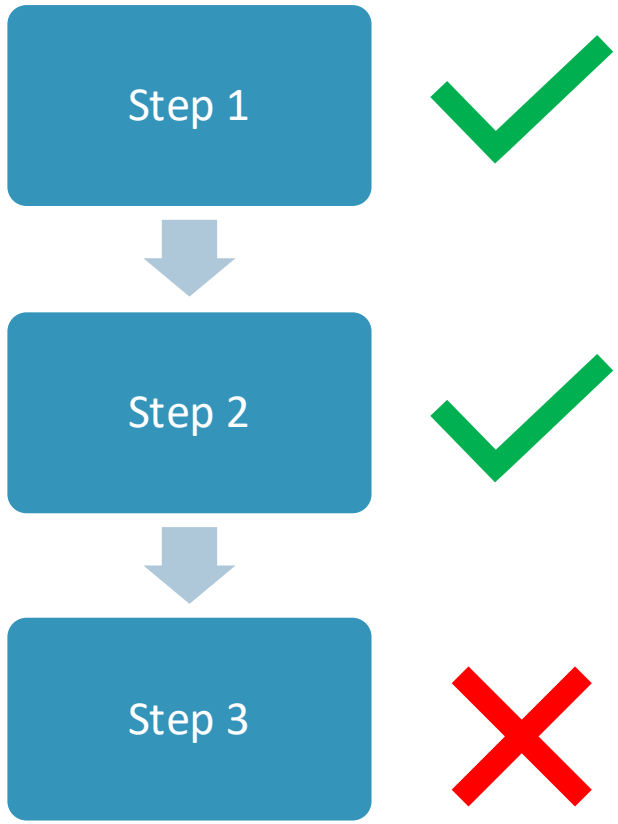
nextflow

Easy construction of a complete pipeline



nextflow

nextflow example.nf

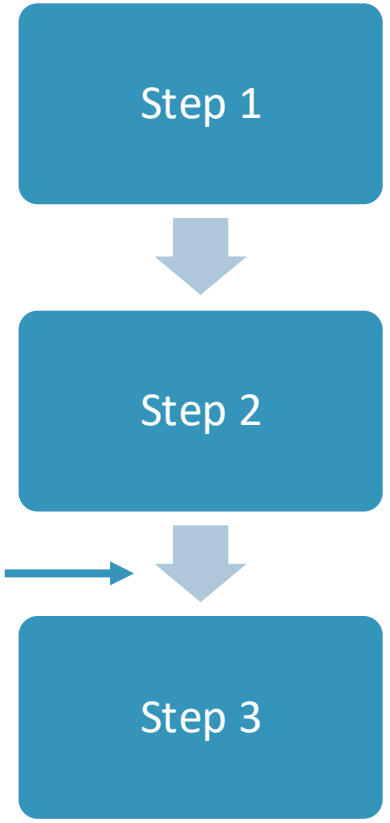


Regular checkpoints

Failure at a single step does not mean you have to rerun the whole pipeline!

- Completed tasks are cached and
- A pipeline can be resumed from the last cached checkpoint

nextflow example.nf -resume



Cached

Cached

Fix the bug → **Resume**

nextflow

Highly Portable

Nextflow pipelines are designed to be highly customizable to your specific files

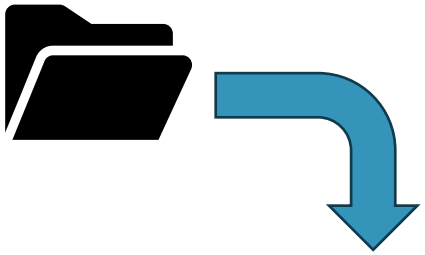
Example.nf

```
process project_merge {

 input:
 path "projection_job_*.ProPC.coord"

 output:
 path "trace.ProPC.coord"

 publishDir "output/", mode: "copy"

 script:
 """
 head -n 1 projection_job_1.ProPC.coord >> trace.ProPC.coord
 for i in projection_job_*.ProPC.coord
 do
 tail -n +2 \$i >> trace.ProPC.coord
 done
 """
}


process infer_ancestry {
  debug true

  input:
  path "reference.RefPC.coord"
  path "trace.ProPC.coord"

  output:
  path "predicted_ancestry.txt"
  stdout emit: rf_log

  publishDir "output/", mode: "copy"

  script:
  """
  rf_model.py -r reference.RefPC.coord -s trace.ProPC.coord -k ${params.nPCs} -p ${params.reference_pop}
  """

}
```

nextflow.config

```
params {
    // requires chr*.vcf.gz format ( with index )
    input_reference = "path/to/reference/chr*.vcf.gz"
    input_study = "path/to/study/chr*.vcf.gz"

    // path to unzipped LASER directory
    path_to_laser = "path/to/LASER-2.04"

    // number of PCs to compute (and then project and infer ancestry)
    nPCs = 20 // default 20

    // reference population file (.csv with ID column and population column)
    reference_pop = "path/to/hgdp_tgp_meta_genetic_region.csv"

    // Minimum probability threshold for assigning population label w RF model
    min_prob = 0.5 // default 0

    // Random seed to use for RF model
    seed = 11

    // Number of jobs to use for parallelization of projection step
    n_jobs = 100
}


process {
        executor = "slurm"
    clusterOptions = "--account="
    cpus = 1
        time = "3d"
        memory = "10GB"
}
```

/bin



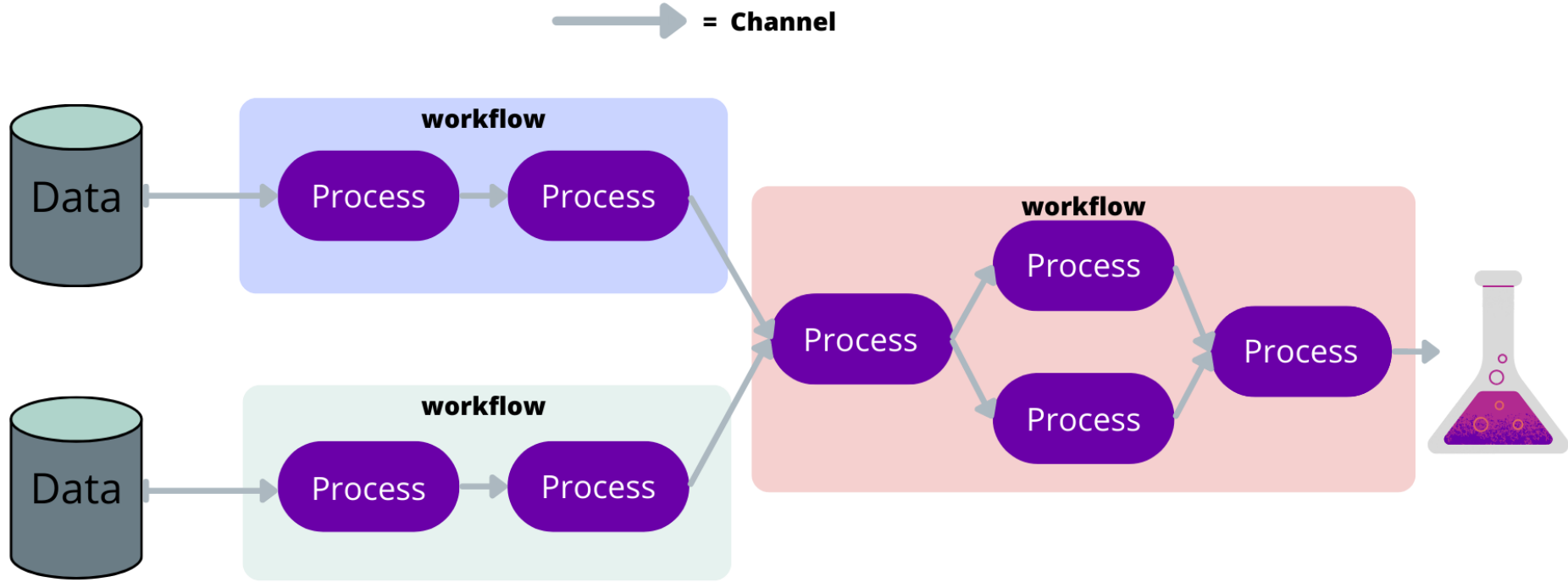
Script1.py
Script2.py

https://github.com/CERC-Genomic-Medicine/HGDP_1KG_ancestry_inference.git

nextflow

Parallelizable

Nextflow pipelines can splinter tasks and run downstream steps while waiting on the splintered tasks to complete



https://carpentries-incubator.github.io/Pipeline_Training_with_Nextflow/02-Intro_to_Nextflow/index.html

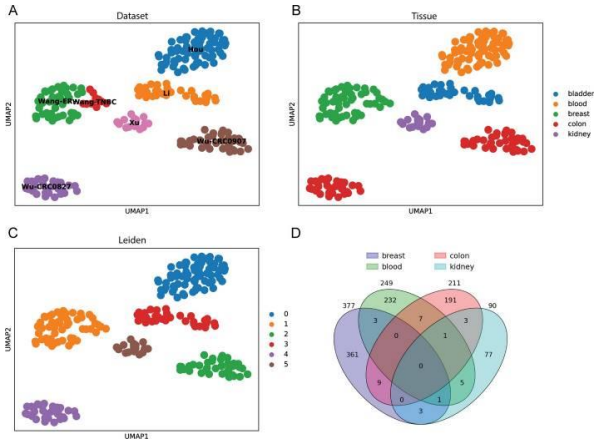
nextflow

Minimizes intermediate file management

Nextflow pipelines, you designate the output of which processes should be sent to your final output directory

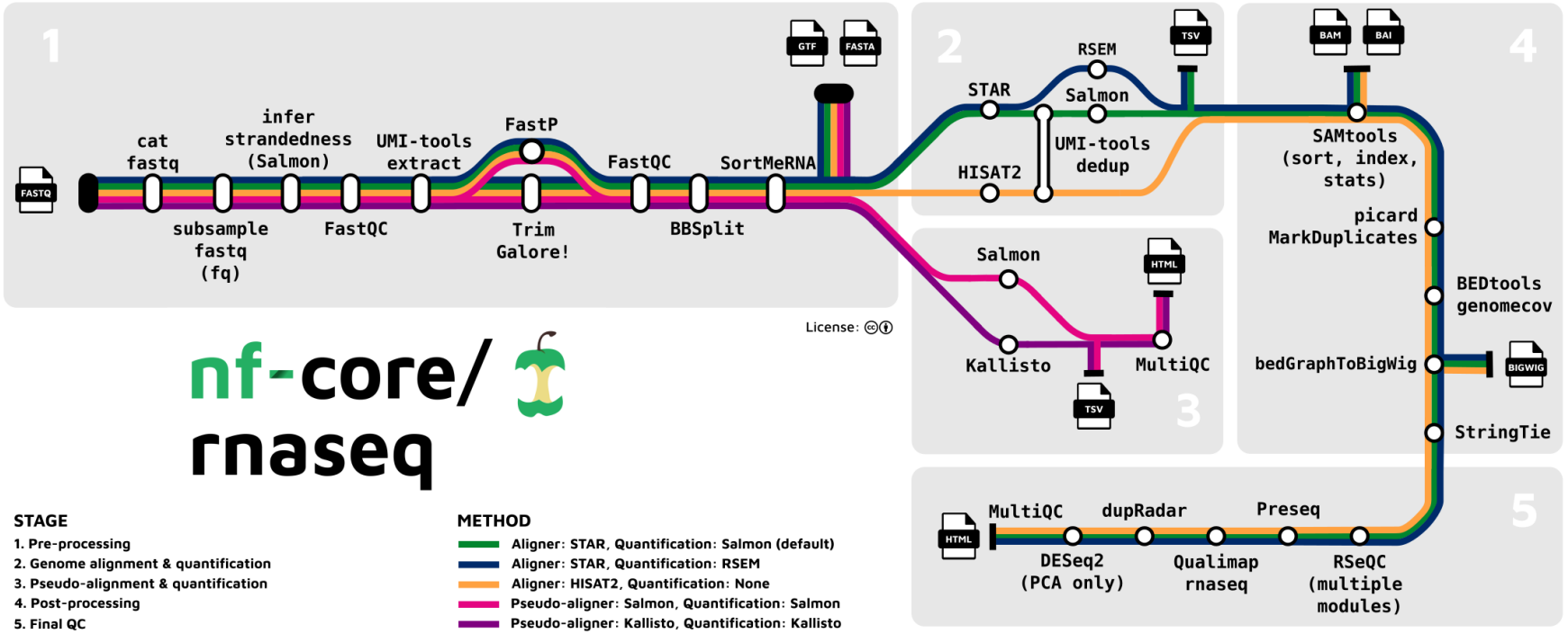
/output

Step 1 → Step 2 → Step 3



10.1186/s12859-024-05928-x

/work



nf-core/rnaseq

STAGE

1. Pre-processing
2. Genome alignment & quantification
3. Pseudo-alignment & quantification
4. Post-processing
5. Final QC

METHOD

- Aligner: STAR, Quantification: Salmon (default)
- Aligner: STAR, Quantification: RSEM
- Aligner: HISAT2, Quantification: None
- Pseudo-aligner: Salmon, Quantification: Salmon
- Pseudo-aligner: Kallisto, Quantification: Kallisto

Prepare the input

- nf-core pipelines require sample metadata and sequencing data
- Requires a samplesheet, comma-separated file with at least columns with sample (GROUP_REPLICATE), fastq_1/fastq_2 or bam, and a header row.

```
sample,fastq_1,fastq_2,strandedness
CONTROL_REP1,AEG588A1_S1_L002_R1_001.fastq.gz,AEG588A1_S1_L002_R2_001.fastq.gz,forward
CONTROL_REP2,AEG588A2_S2_L002_R1_001.fastq.gz,AEG588A2_S2_L002_R2_001.fastq.gz,forward
CONTROL_REP3,AEG588A3_S3_L002_R1_001.fastq.gz,AEG588A3_S3_L002_R2_001.fastq.gz,forward
TREATMENT_REP1,AEG588A4_S4_L003_R1_001.fastq.gz,,reverse
TREATMENT_REP2,AEG588A5_S5_L003_R1_001.fastq.gz,,reverse
TREATMENT_REP3,AEG588A6_S6_L003_R1_001.fastq.gz,,reverse
TREATMENT_REP3,AEG588A6_S6_L004_R1_001.fastq.gz,,reverse
```

Adapted from: https://nf-co.re/rnaseq/3.14.0/

Input configuration files

Input parameters are divided into process (steps) parameters and resource configuration:

Parameters are received with the nextflow command (e.g. genome, alignment and trimming parameters) or by creating a custom configuration file using the nf-core tool online.

```
# Example of JSON config file
cat nf-myparameters.json
{
   "input": "~\/scratch\/nf-test\/samplesheet.csv",
   "outdir": "~\/scratch\/nf-test\/output",
   "trim_fastq": true
}
```

Resource request for each pipeline step can be defined in a .config file

```
# Example cofconfig file
cat nf-mycustom.config
process {
   withName: 'PROCESS_NAME' {
      cpus = 16
      memory = 64.GB
   }
}
```

Execute pipeline

- Nextflow RNA-seq pipeline runs in DRAC but it needs to be submitted as a script.

```
nextflow run \
      nf-core/rnaseq \
      --input <SAMPLESHEET> \
      --outdir <OUTDIR> \
      --fasta <GENOME FASTA> \
      --igenomes_ignore \
      --genome null \
      -profile test,singularity
     -params-file <nf-myparameters_json>
     -c <nf-mycustom_config>
```

Digital Research Alliance of Canada
Alliance de recherche numérique du Canada

```
#!/bin/bash
#SBATCH --time=08:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
module load python/3.11
source nf-core-env/bin/activate
module load apptainer
module load nextflow
export NFCORE_PL=<pipeline_name>
export PL_VERSION=<pipeline_version>
export FD_VERSION=<metadata-format_version>
export NXF_SINGULARITY_CACHEDIR=/project/<def-
group>/NXF_SINGULARITY_CACHEDIR
export SLURM_ACCOUNT=<def-pname>
nextflow run \
      nf-core/rnaseq \
      --input <SAMPLESHEET> \
      --outdir <OUTDIR> \
      --fasta <GENOME FASTA> \
      --igenomes_ignore \
      --genome null \
      -profile test,singularity
      -params-file <nf-myparameters_json>
       -c <nf-mycustom_config>
```

Adapted from: https://nf-co.re/rnaseq/3.14.0/ & https://docs.alliancecan.ca/wiki/Nextflow

Collecting outputs

As jobs finish, multiples folders will be created in the output directory for every analysis step and work directory for intermediate steps and debugging.

To understand the output look at the documentation of each tool in the pipeline.

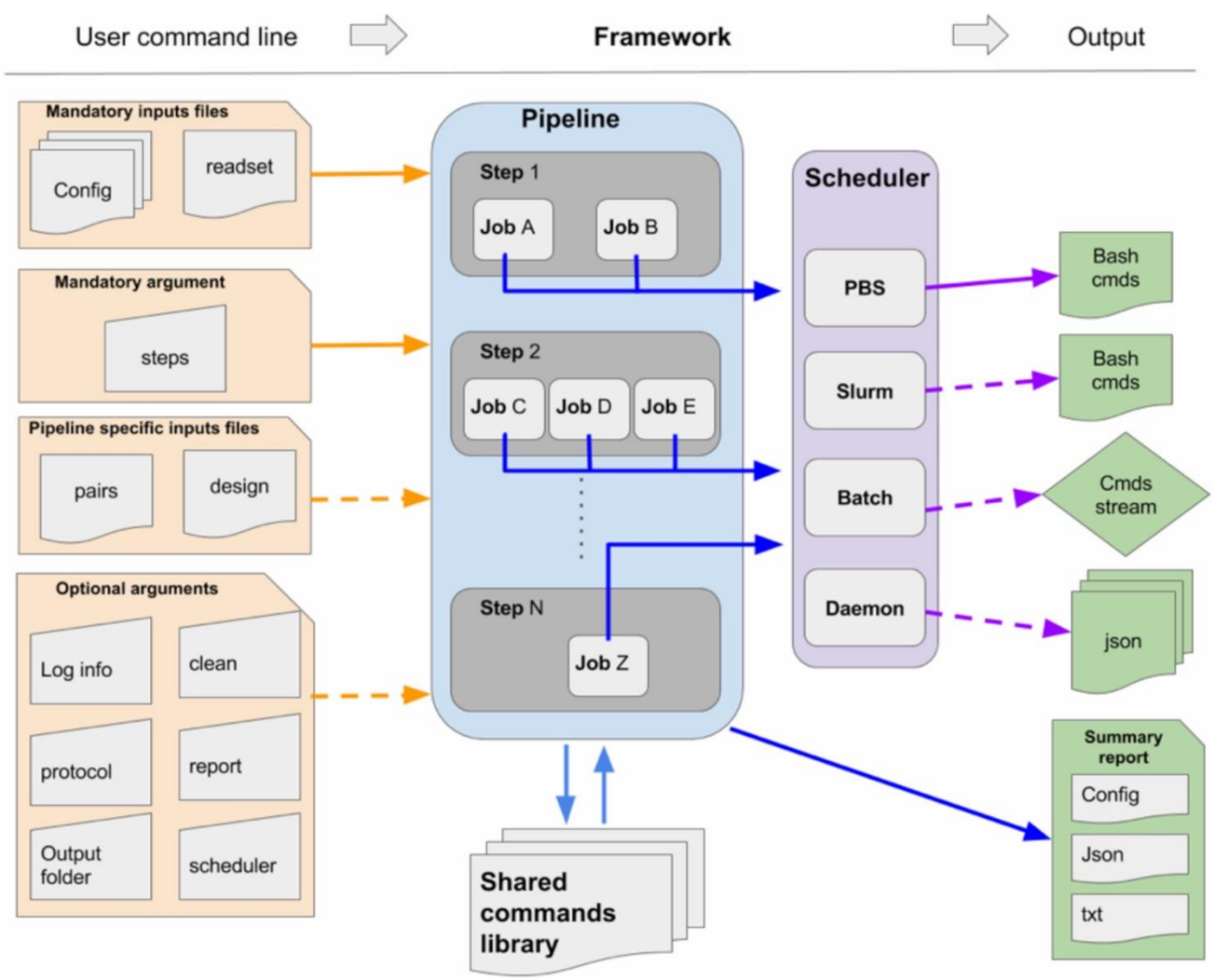
```
output/
 ├── bbsplit
 ├── fastqc
 ├── pipeline_info
|    ├── execution_report_2026-05-27_00-31-47.html
|    ├── execution_timeline_2026-05-27_00-31-47.html
|    ├── execution_trace_2026-05-27_00-31-47.txt
|    ├── nf_core_rnaseq_software_mqc_versions.yml
|    ├── params_2026-05-27_00-32-11.json
|    └── pipeline_dag_2026-05-27_00-31-47.html
…
Work/
 ├── da
|    └── 32a31d0980f1275f9abb2621f22584
|        ├── H1ESC_Rep1_raw_1_fastqc.html
|        ├── H1ESC_Rep1_raw_1.gz -> rnaseq_H1ESC_chr19_Rep1_R1.fastq.gz
|        ├── H1ESC_Rep1_raw_2_fastqc.html
```

GenPipes

- Validated NGS pipelines - C3G-maintained with vetted community contribution. support@computationalgenomics.ca
- HPC-optimized
- Distributed in DRAC
- Low barrier to entry
- Python based
- Wizard helps you choose the pipeline

https://genpipes.readthedocs.io/en/genpipes-v6.1.1/index.html

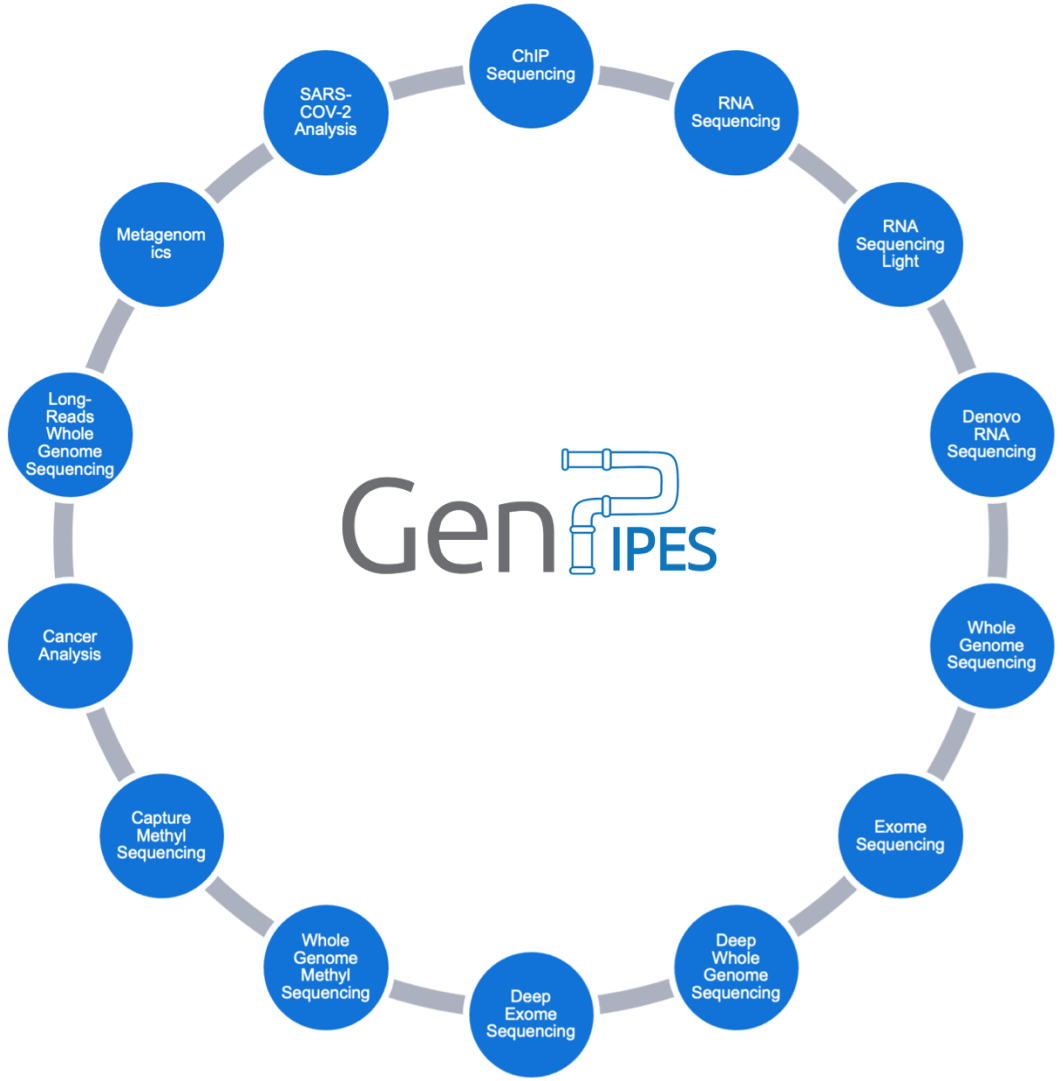
GenPIPES



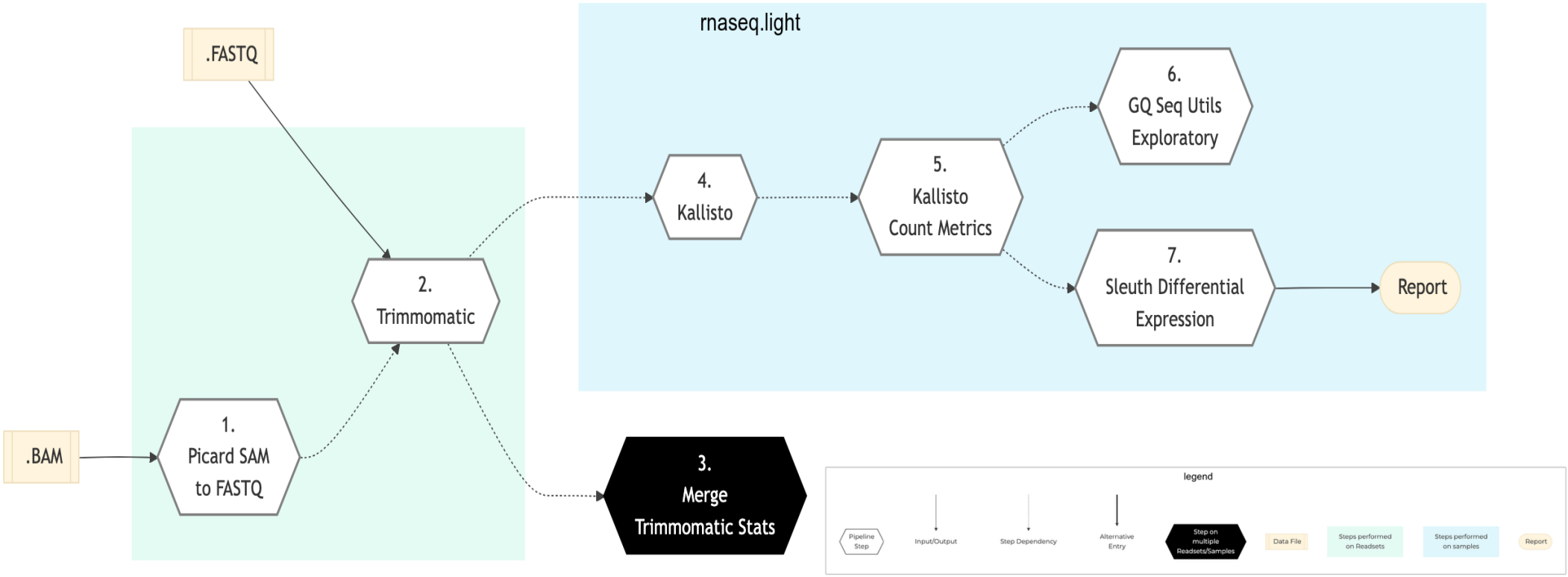
Bourgey et al., 2019

GenPipes – 10 genomic pipelines

- Ampliconseq
- covseq
- Methylseq
- rnaseq_denovo_assembly
- chipseq
- dnaseq
- longread_dnaseq
- nanopore_covseq
- **rnaseq**
- **rnaseq_light**



rnaseq_light pipeline



All pipelines require 2 files:

Configuration ".ini" file: contains parameters used by tools in the pipeline

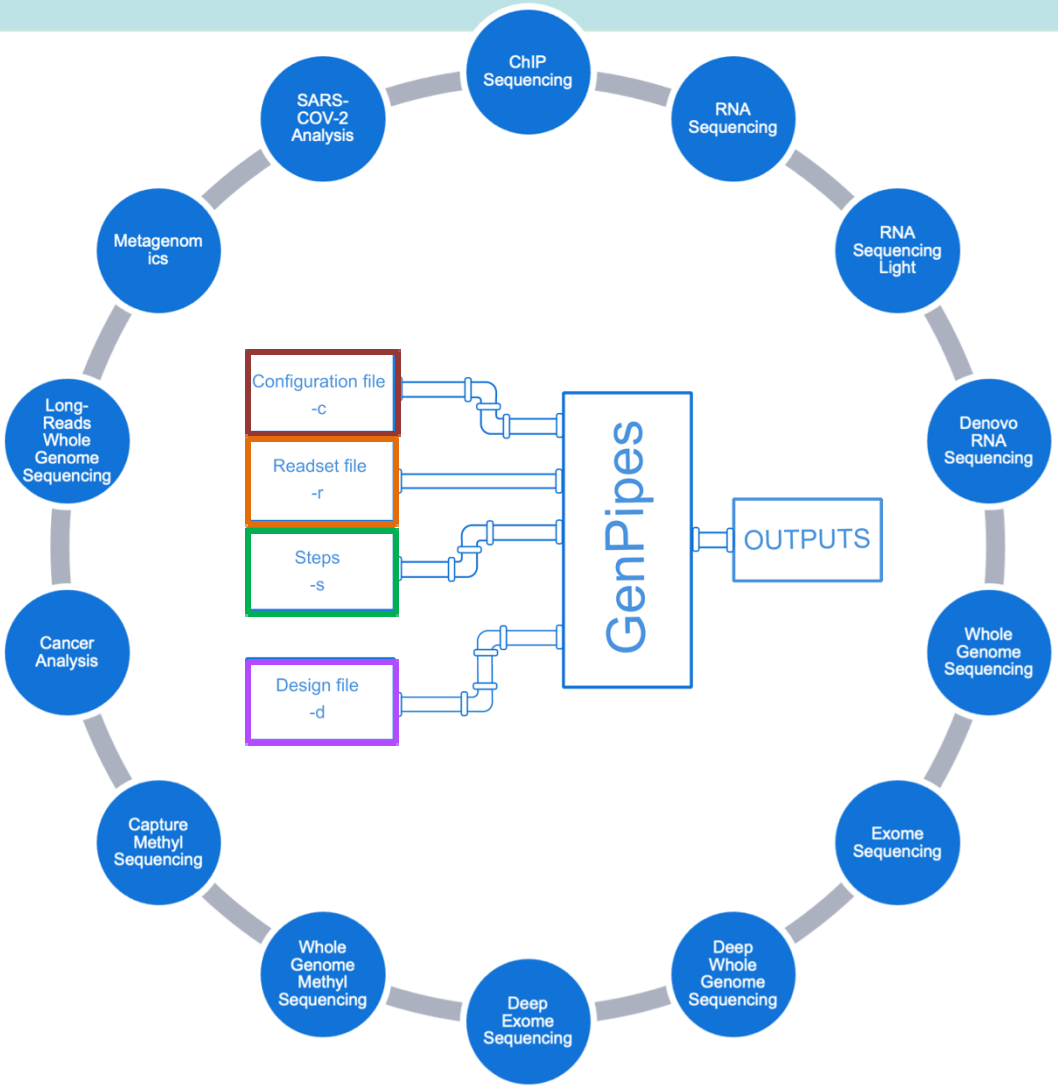
Readset file: contains details about your samples

Optional:

Steps selection: defines the steps of the pipeline to be run

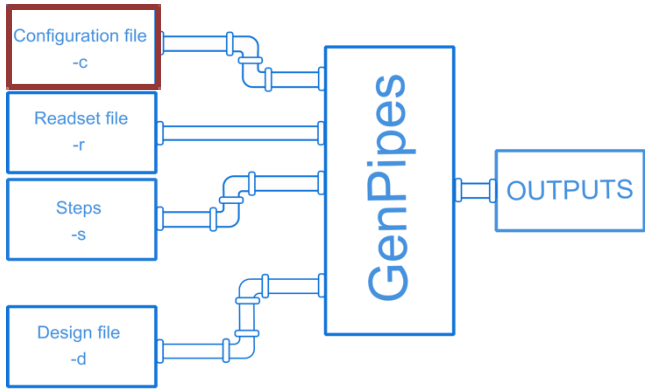
A few pipelines require:

Design file: contains details about sample comparisons (e.g. RNA-seq, ChiP-seq, etc…)

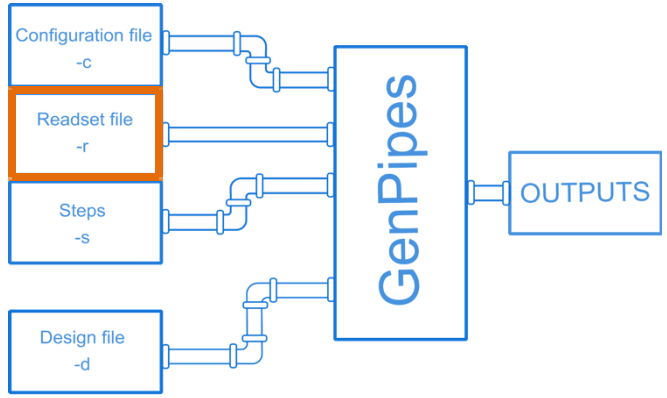


Configuration/ ini files

- Configuration files also know as “ini” files due to their file extension.
- Provides parameters for each step of the pipeline in a single file – no necessary to type them in the command line.
- GenPipes provide a default ini files for each pipeline, DRAC server and for different species.



Readset file

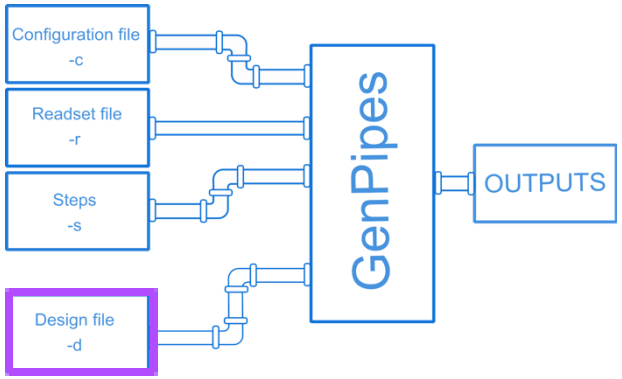


- Describes the sequencing data and to which sample it belongs to:
  - Readset used for analysis
  - Samples to be merged
  - Location of FASTQ/BAM input files.
- The readset file contains the following **tab-separated** fields:

Optional | Optional | Optional | Mandatory if FASTQs aren't available

Sample	Readset	Library	RunType	Run	Lane	Adapter1	Adapter2	QualityOffset	BED	FASTQ1	FASTQ2	BAM
sampleA	readset1	lib0001	PAIRED_END	run100	1	AGATCGGAAGAGCACAC	AGATCGGAAGAGC	33	path/to/file.bed	path/to/r1.paired1.fastq.gz	path/to/r1.paired2.fastq.gz	path
sampleA	readset2	lib0001	PAIRED_END	run100	2	AGATCGGAAGAGCACAC	AGATCGGAAGAGC	33		path/to/file.bed	path/to/r2.paired1.fastq.gz	path/to/r2.paired2.fa
sampleB	readset3	lib0002	PAIRED_END	run200	5	AGATCGGAAGAGCACAC	AGATCGGAAGAGC	33		path/to/file.bed	path/to/r3.paired1.fastq.gz	path/to/r3.paired2.fa
sampleB	readset4	lib0002	PAIRED_END	run200	6	AGATCGGAAGAGCACAC	AGATCGGAAGAGC	33		path/to/file.bed	path/to/r4.paired1.fastq.gz	path/to/r4.paired2.fa

Design Files



- It contains data about sample comparisons.

- It requires 2 fields which are **tab-separated**:
  - -1st column contains samples: sample name must match a sample name in the readset file
  - - Other columns contain contrast: defines an experimental design contrast.

'0' or '': the sample is not included in the analysis
'1': the sample belongs to the control group
'2': the sample belongs to the treatment test case group

Design file

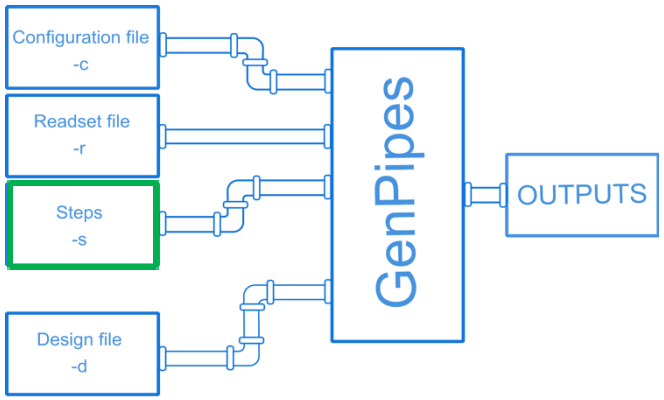
Sample	Contrast1	Contrast2
sampleA	1	1
sampleB	2	0
sampleC	0	2

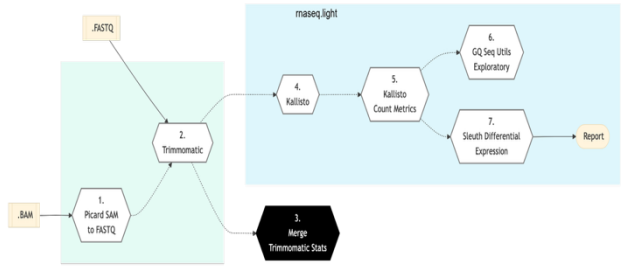
Readset file

Sample	Readset	Library	RunType	Run	Lane
sampleA	readset1	lib0001	PAIRED_END	run100	1
sampleA	readset2	lib0001	PAIRED_END	run100	2
sampleB	readset3	lib0002	PAIRED_END	run200	5
sampleB	readset4	lib0002	PAIRED_END	run200	6
sampleC	readset5	lib0002	PAIRED_END	run200	6
sampleC	readset6	lib0002	PAIRED_END	run200	6

Steps

- A pipeline has many steps and the steps parameter tells the genpipes command which steps to run.
- To check the steps inside a pipeline, use:
- By default GenPipes run all the steps but you can specify only to run a subset (e.g. steps 1 to 5 and 8).
- CACHE is saved for each step to allow fixing bugs and restarting pipeline.





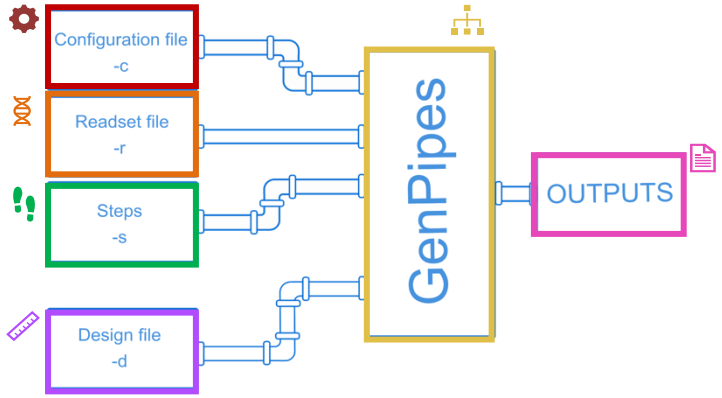

```
genpipes rnaseq_light -h
```

Protocol

1. picard_sam_to_fastq
2. Trimmomatic
3. merge_trimmomatic_stats
4. Kallisto
5. kallisto_count_matrix
6. gq_seq_utils_exploratory_analysis_rnaseq_light
7. sleuth_differential_expression
8. multiqc

Create GenPipes command script

Genpipes command generates the script to run the pipeline. It **DOES NOT** run the pipeline, it creates dependency structure.




```
# Example of running genpipes command
genpipes rnaseq_light
            -c rnaseq.base.ini\
            narval.ini \
            Mus_musculus.GRCm38.ini\
            rnaseq.myparameters.ini\
            -r readset.tsv \
            -s 1-5,8 \
            -d design.tsv \
            -g rnaseq_light_genpipes.sh\
            -o /output/directory \


bash rnaseq_light_genpipes.sh
```

```
# Default pipeline configuration file
# Server configuration file
# Genome configuration file
# Custom configuration file
# Readset file
# Run only steps 1 to 7, and 12 of the pipeline
# Design file for condition comparison
# Name of output GenPipes script
# Directory to write pipeline output (in scratch)
```

Collect outputs and reports

As jobs finish, log files will be created in the “job_output” folder.

For every analysis step, a folder will be created with the appropriate output files. Look into all the folders to know what each pipeline produces.

```
Results
├── job_output
│   └── gq_seq_utils_exploratory_analysis_rnaseq_light
│       ├── gq_seq_utils_exploratory_analysis_rnaseq_light.06e5541f51ce0e1e478c419d88a2a097.mugqic.done
│       ├── gq_seq_utils_exploratory_analysis_rnaseq_light_2026-05-24T23.02.09.o
│       ├── gq_seq_utils_exploratory_analysis_rnaseq_light_2026-05-24T23.02.09.sh
│       └── gq_seq_utils_exploratory_analysis_rnaseq_light_2026-05-24T23.02.09.submit
.
.
.
├── report
│   └──RnaSeqLight.multiqc.html
├── exploratory
├── kallisto
├── report
├── RnaSeqLight.default.2026-05-24T23.02.09.config.trace.ini
```

Hands on – creating input files

Training resources and support:

MiCM: https://www.mcgill.ca/micm/

C3G: http://www.computationalgenomics.ca/open-door/

MUCH bioinformatics: https://bioinfo.rimuhc.ca/labkey/home/project-begin.view?pageId=LabKey

HeDS: https://hedscenter.ca/en

CDSI: https://www.mcgill.ca/cdsi/training